REAL-TIME OPERATING SYSTEM

University of Alexandria, Faculty of Engineering,
Computer and Systems Engineering Department.
CSE343 Embeded Systems Spring 2026

Real-Time System

- Real-time systems are designed to do something within a certain amount of time; they guarantee that stuff happens when it's supposed to.
- A pacemaker is an excellent example of a real-time embedded system.
  - A pacemaker must contract the heart muscle at the right time to keep you alive; it can't be too busy to respond in time.
  - Pacemakers and other real-time embedded systems are carefully designed to run their tasks on time, every time.

Real-Time Operating System

- Guarantees real-time applications.
- Commonly used in embedded systems.
- Provides the following functionality:
  - Multitasking
    - Tasks are rapidly switched between to give the impression that multiple programs are executing concurrently;
  - Prioritization
  - Interrupt levels

Real-Time Operating System Characteristics

- **Small footprint.** Compared to general OSes, real-time operating systems are lightweight.
- **High performance.** RTOSes are typically fast and responsive.
- **Determinism.** Repeating inputs end in the same output.
- **Safety and security.** Safety-critical and security standards are typically the highest priority, as RTOSes are frequently used in critical systems.
- **Priority-based scheduling.** Tasks that are assigned a high priority are executed first followed by lower-priority jobs.
- **Timing information.** RTOSes are responsible for timing and providing application programming interface (API).

FREE RTOS

- FreeRTOS Real Time Operating System implemented for AVR (Uno, Nano, Leonardo, Mega).

Review: Kernel

- Kernel is the core component of an operating system that manages operations of computer and hardware.
  - memory and CPU time.

API Interface

- Task Creation
- Task Control
- Task Utilities
- Queue Management
- Semaphores

Task Creation

- xTaskCreate
  - Create a new task and add it to the list of tasks that are ready to run.
- vTaskDelete
  - Remove a task from the RTOS kernels management. The task being deleted will be removed from all ready, blocked, suspended and event lists.

Task Control

- vTaskDelay
  - Delay a task for a given number of ticks. The actual time that the task remains blocked depends on the tick rate.
- vTaskSuspend
  - Suspend any task. When suspended a task will never get any microcontroller processing time, no matter what its priority.
- vTaskResume
  - Resumes a suspended task.
  - A task that has been suspended by one or more calls to vTaskSuspend () will be made available for running again by a single call to vTaskResume ().

Task Utilities

- xTaskGetCurrentTaskHandle
  - The handle of the currently running (calling) task.
- uxTaskGetStackHighWaterMark
  - The stack used by a task will grow and shrink as the task executes and interrupts are processed. uxTaskGetStackHighWaterMark() returns the minimum amount of remaining stack space that was available to the task since the task started executing - that is the amount of stack that remained unused when the task stack was at its greatest (deepest) value. This is what is referred to as the stack 'high water mark'.
- pcTaskGetName
  - A pointer to the subject task's name, which is a standard NULL terminated C string.
- uxTaskGetNumberOfTasks
  - The number of tasks that the RTOS kernel is currently managing. This includes all ready, blocked and suspended tasks. A task that has been deleted but not yet freed by the idle task will also be included in the count.

Queue Management

- xQueueCreate
  - Creates a new queue and returns a handle by which the queue can be referenced.
- xQueueSend
  - Post an item on a queue. The item is queued by copy, not by reference.
- xQueueReceive
  - Receive an item from a queue. The item is received by copy so a buffer of adequate size must be provided. The number of bytes copied into the buffer was defined when the queue was created.

Semaphores

- xSemaphoreCreateMutex
  - Creates a mutex, and returns a handle by which the created mutex can be referenced. Mutexes cannot be used in interrupt service routines.
- xSemaphoreTake
  - Obtains a semaphore. The semaphore must have previously been created.
- xSemaphoreGive
  - Releases a semaphore. The semaphore must have previously been created.

Compilation Statistics

```
FreeRTOS Library Files Footprints
=================================

 Volume in drive C has no label.
 Volume Serial Number is 08DC-D539

 Directory of C:\Users\salah\AppData\Local\Temp\arduino\sketches\CE41AC306EC2C925E1BD572EDBCAA328\libraries\FreeRTOS

04/18/2024  09:52 PM            20,424 event_groups.c.o
04/18/2024  09:52 PM             5,384 heap_3.c.o
04/18/2024  09:52 PM             8,440 list.c.o
04/18/2024  09:52 PM            13,440 port.c.o
04/18/2024  09:52 PM            57,640 queue.c.o
04/18/2024  09:52 PM            41,568 stream_buffer.c.o
04/18/2024  09:52 PM           108,512 tasks.c.o
04/18/2024  09:52 PM            37,296 timers.c.o
04/18/2024  09:52 PM             8,624 variantHooks.cpp.o
               9 File(s)        301,328 bytes
               0 Dir(s)  18,755,923,968 bytes free
```

Compilation Statistics(2)

```
TaskUtilities.ino
=================
Sketch uses 9068 bytes (28%) of program storage space. Maximum is 32256 bytes.
Global variables use 465 bytes (22%) of dynamic memory, leaving 1583 bytes for local variables. Maximum is 2048 bytes.

TaskStatus.ino
==============
Sketch uses 8760 bytes (27%) of program storage space. Maximum is 32256 bytes.
Global variables use 379 bytes (18%) of dynamic memory, leaving 1669 bytes for local variables. Maximum is 2048 bytes.

ArrayQueue.ino
==============
Sketch uses 9806 bytes (30%) of program storage space. Maximum is 32256 bytes.
Global variables use 409 bytes (19%) of dynamic memory, leaving 1639 bytes for local variables. Maximum is 2048 bytes.

AnalogRead_DigitalRead.ino
==========================
Sketch uses 10456 bytes (32%) of program storage space. Maximum is 32256 bytes.
Global variables use 369 bytes (18%) of dynamic memory, leaving 1679 bytes for local variables. Maximum is 2048 bytes.
```

References

- RTOS
- FreeRTOS API